

BudgetFlowFusion

Dokumentacja analityczno-projektowa i użytkowa

Zakres dokumentu

- analiza dziedziny i projektu
- architektura aplikacji
- diagramy i opis modelu danych
- procesy biznesowe
- API backendu
- instrukcja użytkownika
- uruchomienie, testy i utrzymanie

Spis treści

Dokumentacja BudgetFlowFusion
Wprowadzenie
Dokumentacja analityczno-projektowa
Architektura systemu
Model danych
Procesy biznesowe
API backendu
Dokumentacja użytkownika
Uruchomienie i testy
Utrzymanie i słownik pojęć

Wprowadzenie

Cel projektu

BudgetFlowFusion powstał po to, żeby zebrać w jednym miejscu proces, który w kole naukowym łatwo rozchodzi się po wiadomościach, arkuszach i ustaleniach ze skarbnikiem. Członkowie koła zgłaszają rzeczy do kupienia, skarbnik pilnuje finansowania, a na końcu trzeba jeszcze mieć porządek w wnioskach, planie zamówień publicznych i fakturach.

Aplikacja prowadzi ten proces od katalogu przedmiotów i list zakupów, przez wybór dofinansowania oraz pozycji CPV, aż do finalizacji wniosku i rozliczenia. Najważniejsze jest zachowanie śladu: skąd pochodziły pieniądze, do którego planu podpisano zakup, kto dodał pozycje do listy i na jakim etapie jest wniosek.

Kontekst działania

Koło naukowe może mieć kilka sekcji albo projektów. Każda sekcja ma budżet wynikający z dofinansowań. Dofinansowanie jest konkretną pulą pieniędzy, a nie tylko opisem w tabeli. Dla takiej puli można przygotować plan zamówień publicznych, czyli listę kodów CPV i kwot, które mają być wydane w danym roku.

Później, kiedy skarbnik chce realnie coś kupić, tworzy wniosek o zamówienie. Wniosek powinien wskazywać, z którego dofinansowania korzysta i którą pozycję planu CPV wykorzystuje. Jeżeli zakup nie mieści się w planie albo przekracza zaplanowaną kwotę, system nie ukrywa tego problemu, tylko wymaga uzasadnienia.

Użytkownicy systemu

Zwykły członek koła Dodaje przedmioty, przegląda katalog i może uzupełniać otwarte koszyki zakupowe przypisane do wniosków dostępnych dla jego koła.

Skarbnik Zarządza koszykami, budżetami, dofinansowaniami, planami publicznymi, wnioskami, finalizacją oraz rozliczeniami.

Osoba rozliczająca / księgowa w procesie W obecnym systemie jej czynności są reprezentowane głównie przez moduł rozliczeń, faktury, statusy i kontrolę wykorzystania pozycji planu.

Główne założenia

- Jedno dofinansowanie należy do jednej sekcji/projektu.
- Budżet sekcji jest sumą dofinansowań tej sekcji.
- Budżet koła jest sumą budżetów sekcji.
- Plan zamówień publicznych jest przypisany do dofinansowania.
- Pozycja planu publicznego opisuje kod CPV i kwotę planowaną.
- Wniosek może korzystać z jednej albo wielu alokacji finansowania.
- Wniosek może wykorzystywać jedną albo wiele pozycji planu.
- Finalizacja wniosku zamyka część zakupową i tworzy dane do rozliczeń.

Zakres techniczny aplikacji

BudgetFlowFusion jest aplikacją typu SPA + REST API. Frontend nie posiada własnej bazy ani warstwy offline. Wszystkie dane operacyjne są pobierane z backendu przez endpointy /api/....

```

Vue component
|
| fetch(JSON)
v
FastAPI endpoint
|
| SQLAlchemy Session
v
PostgreSQL table

```

Backend nie ma oddzielnej warstwy serwisowej. Reguły domenowe są zaimplementowane głównie w plikach z trasami:

- public_purchase_plans_routes.py - budżety, dofinansowania i plan zamówień publicznych,
- purchase_request_routes.py - wnioski, alokacje, finalizacja i linie rozliczeniowe,
- lists_routes.py - koszyki sklepowe, wkład studentów i zamykanie list,
- settlements_routes.py - rozliczenia, faktury i historia,
- items_routes.py - wspólny katalog przedmiotów,
- fundings_routes.py - dofinansowania i zadania budżetowe.

Źródła prawdy danych finansowych

Aktualne kwoty nie powinny być interpretowane jako pojedyncze pole zapisane w jednej tabeli. System wylicza je dynamicznie z kilku tabel.

```

Budżet dofinansowania:
    Funding.funding_price

Wydane z dofinansowania:
    Funding.spent_money

Zarezerwowane we wnioskach:
    SUM(PurchaseRequestFundingAllocation.allocated_amount)
    albo legacy fallback:
    SUM(PurchaseRequest.budget_allocated_for_the_order)

Dostępne po wnioskach:
    funding_price - spent_money - reserved

Budżet sekcji:
    SUM(Funding.funding_price WHERE project_budget_id = X)

Budżet koła:
    SUM(budżetów sekcji dla projektów danego Association)

```

Najważniejsza konsekwencja projektowa: ProjectBudget.total_budget i AssociationBudget.total_budget są synchronizowane i raportowane na podstawie dofinansowań. Nie należy traktować ich jako niezależnych, ręcznie ustawianych kwot.

Granice odpowiedzialności ról

Rola użytkownika wynika z relacji Student.project_finance_manager_id. Jeżeli to pole jest ustawione, frontend traktuje użytkownika jako skarbnika. Jeżeli jest puste, użytkownik jest zwykłym członkiem koła.

W obecnej implementacji autoryzacja jest uproszczona: frontend zapisuje zalogowanego użytkownika w localStorage, a backend przyjmuje student_id, association_id albo project_finance_manager_id w parametrach zapytania lub payloadzie. Dla operacji finansowych część endpointów wykonuje dodatkowe sprawdzenia, np. czy student jest skarbnikiem lub czy dofinansowanie należy do koła użytkownika.

Dokumentacja analityczno-projektowa

Cel analizy

W projekcie najważniejsze było uchwycenie zależności, które przy zakupach w kole naukowym często są rozproszone. Przedmiot na liście to tylko początek. Trzeba jeszcze wiedzieć, z jakiego finansowania będzie opłacony, czy pasuje do planu zamówień publicznych, jaką część planu zużywa i czy później da się go rozliczyć fakturą.

Z tego powodu model aplikacji został ułożony wokół kilku prostych zasad:

- pieniądze w systemie pochodzą z dofinansowania,
- dofinansowanie należy do jednej sekcji/projektu,
- sekcje składają się na budżet koła,
- każde dofinansowanie może mieć własny plan zamówień publicznych,
- pozycja planu opisuje kod CPV i kwotę zaplanowaną na przyszły zakup,
- wniosek o zamówienie zużywa środki z dofinansowania i pozycje planu,
- rozliczenie potwierdza zakup fakturą i kwotą rzeczywistą.

Kontekst organizacyjny

```
Koło naukowe
|
+-- członkowie
|
+-- skarbnicy
|
+-- sekcje/projekty
    |
    +-- budżety sekcji
        |
        +-- dofinansowania
            |
            +-- plany zamówień publicznych
            |
            +-- listy zakupów
            |
            +-- wnioski
                |
                +-- finalizacja
                |
                +-- rozliczenie
                    |
                    +-- faktury
```

System nie modeluje pełnego obiegu dokumentów uczelni. Modeluje tę część procesu, którą skarbnik koła musi mieć pod kontrolą: dostępne środki, uzasadnienie wydatku, zgodność z planem CPV, historię wniosku i stan rozliczenia.

Aktorzy

Zwykły członek koła

Zwykły członek pracuje przede wszystkim na katalogu i otwartych listach zakupów. Może dodać przedmiot do wspólnego katalogu, dopisać swoją ilość do listy zakupów oraz usunąć wkład,

który sam dodał.

W modelu danych jest to rekord Student bez project_finance_manager_id.

Skarbnik

Skarbnik odpowiada za finanse koła. Tworzy dofinansowania, plany publiczne, listy zakupów, wnioski, finalizuje wnioski i prowadzi rozliczenia.

W modelu danych jest to Student połączony z ProjectFinanceManager przez Student.project_finance_manager_id.

Osoba rozliczająca

W aplikacji nie jest osobną rolą logowania. Jej praca jest reprezentowana przez moduł rozliczeń: faktury, linie rozliczenia, statusy faktur i status settled na wniosku.

Główne obiekty dziedziny

Association

Koło naukowe. Granica danych użytkowników, projektów i budżetów.

Student

Użytkownik aplikacji. Może być zwykłym członkiem albo skarbnikiem.

Project

Sekcja lub projekt działający w ramach koła.

ProjectBudget

Budżet sekcji. Kwota wynika z dofinansowań przypiętych do sekcji.

AssociationBudget

Budżet całego koła. Jest sumą budżetów sekcji.

Funding

Konkretne dofinansowanie. Jest źródłem pieniędzy dla zakupów.

PublicPurchasePlanList

Roczny plan zamówień publicznych dla jednego dofinansowania.

PublicPurchasePlan

Pozycja planu: kod CPV, numer pozycji, opis i planowana kwota.

ShopPurchaseList

Lista zakupów dla jednego sklepu w ramach grupy zakupowej.

Item

Przedmiot w katalogu zakupowym.

PurchaseRequest

Wniosek o zamówienie. Łączy środki, CPV, listy zakupów i statusy.

Settlement

Rozliczenie wniosku lub zamkniętej listy.

Invoice

Faktura przypisana do rozliczenia.

Decyzje projektowe

Budżet jest liczony od dofinansowań

Budżet sekcji nie jest niezależną kwotą wpisaną ręcznie. Sekcja ma tyle środków, ile wynosi suma jej dofinansowań.

```
Funding 1 10 000 PLN
Funding 2 5 000 PLN
|
v
ProjectBudget 15 000 PLN
```

Analogicznie budżet koła jest sumą budżetów sekcji.

Plan publiczny należy do dofinansowania

Plan zamówień publicznych nie jest wspólny dla całego koła. Jest powiązany z dofinansowaniem, ponieważ to dofinansowanie określa, z jakiej puli pieniędzy zakup będzie finansowany.

```
Funding
|
| 1 : 0..1
v
PublicPurchasePlanList
|
| 1 : *
v
PublicPurchasePlan
```

Pozycja planu nie jest konkretnym koszykiem zakupowym. Opisuje zamiar: kod CPV i kwotę planowaną na zakupy w danym kodzie.

Wniosek zużywa plan i rezerwuje środki

Wniosek o zamówienie jest dokumentem operacyjnym: skarbnik chce zrealizować zakup teraz. Wniosek wskazuje dofinansowanie i pozycje planu, z których korzysta. Kwota wniosku zmniejsza dostępny budżet już na etapie złożenia, ponieważ pieniądze są traktowane jako zarezerwowane.

```
Funding.funding_price
- Funding.spent_money
- SUM(PurchaseRequestFundingAllocation.allocated_amount)
= available_after_purchase_requests
```

Koszyk jest roboczą listą zakupów

Lista zakupów jest roboczym koszykiem dla jednego sklepu. Do jednej grupy zakupowej może należeć wiele koszyków, np. gdy wniosek obejmuje zakupy z kilku sklepów.

```
GroupedShopsListByCpvCategoryAndFunding
|
+-- ShopPurchaseList: sklep A
+-- ShopPurchaseList: sklep B
+-- ShopPurchaseList: sklep C
```

Użytkownicy dopisują do koszyka swoje ilości. Łączna ilość jest w ShopPurchaseListItem, a informacja kto ile dodał jest w ShopPurchaseListItemContribution.

Finalizacja zapisuje stan historyczny

Plan, dofinansowanie i koszyk mogą później zostać zmienione. Dlatego finalizacja tworzy

PurchaseRequestFinalizationSnapshot. Snapshot zapisuje dane potrzebne do dokumentu: CPV, numer planu, osobę odpowiedzialną, finansowanie i kwoty.

```
PurchaseRequest
|
| finalize
v
PurchaseRequestFinalizationSnapshot
|
+-- dane CPV
+-- dane planu
+-- dane dofinansowania
+-- kwoty netto, brutto i EUR
```

Granice systemu

System obejmuje:

- użytkowników koła,
- katalog przedmiotów i sklepów,
- listy zakupów,
- dofinansowania i zadania budżetowe,
- plany zamówień publicznych,
- wnioski o zamówienie,
- finalizację wniosku,
- rozliczenia i faktury.

System nie obejmuje w pełni:

- podpisu elektronicznego,
- integracji z SAP,
- automatycznego pobierania faktur,
- oficjalnego obiegu akceptacji uczelnianej,
- wersjonowanych migracji bazodanowych typu Alembic,
- pełnej autoryzacji tokenowej.

Mapa odpowiedzialności modułów

Moduł frontendowy	Odpowiedzialność
HomePage.vue	pulpit, budżet, nawigacja
AddedItems.vue	katalog przedmiotów
AddedShopPurchaseLists.vue	listy zakupów
ShopPurchaseListDetails.vue	pozycje listy i wkłady studentów
PublicPurchasePlans.vue	dofinansowania i plany CPV
PurchaseRequest.vue	wnioski i finalizacja
Settlement.vue	rozliczenia i faktury
Shops.vue	sklepy

Moduł backendowy	Odpowiedzialność
relations.py	model relacyjny
public_purchase_plans_routes.py	budżety, dofinansowania, plany

lists_routes.py
purchase_request_routes.py
settlements_routes.py
items_routes.py
shops_routes.py
login_register_routes.py

koszyki i wkłady użytkowników
wnioski, alokacje, finalizacja
rozliczenia, faktury, historia
katalog przedmiotów
sklepy
logowanie i rejestracja

Stany najważniejszych obiektów

Shop.status
pending -> approved
pending -> rejected

Item.status
approved
pending -> approved
pending -> rejected

ShopPurchaseList
settlement_id = NULL lista otwarta
settlement_id != NULL lista zamknięta

PurchaseRequest.finalization_status
draft -> prepared -> accounting_pending -> settlement -> settled

PurchaseRequest.plan_compliance_status
draft
compliant
requires_approval

Najważniejsze reguły spójności

1. Dofinansowanie musi należeć do wybranego budżetu sekcji.
2. Plan publiczny jest przypisany do dokładnie jednego dofinansowania.
3. W ramach jednego planu nie powinno być dwóch pozycji z tym samym CPV.
4. Wniosek nie może przekroczyć dostępnych środków dofinansowania.
5. Wniosek nie może przekroczyć dostępnego budżetu sekcji.
6. Przekroczenie pozycji planu wymaga uzasadnienia.
7. Lista zamknięta nie przyjmuje nowych pozycji.
8. Student może usunąć z listy tylko własny wkład.
9. Zakończenie rozliczenia wymaga faktury na każdej linii rozliczenia.
10. Finalizacja wniosku zapisuje snapshot danych historycznych.

Architektura systemu

Widok logiczny

System składa się z trzech głównych warstw:

- frontend Vue,
- backend FastAPI,
- baza danych PostgreSQL.

Diagram komponentów



Backend

Backend znajduje się w katalogu backend. Główne pliki:

- backend/run.py - punkt uruchomienia serwera Uvicorn,
- backend/src/__init__.py - konfiguracja aplikacji FastAPI, CORS, połączenie z bazą, inicjalizacja i migracje,
- backend/src/relations.py - modele danych SQLAlchemy,
- backend/src/routes - endpointy API.

Moduły tras:

- login_register_routes.py - logowanie i rejestracja,
- items_routes.py - przedmioty,
- categories_subcategories_routes.py - kategorie i podkategorie,
- shops_routes.py - sklepy,

- lists_routes.py - koszyki/listy zakupów,
- fundings_routes.py - dofinansowania i zadania,
- public_purchase_plans_routes.py - budżety, plany publiczne i dashboard budżetowy,
- purchase_request_routes.py - wnioski, alokacje, finalizacja i linie rozliczeń,
- settlements_routes.py - rozliczenia i faktury.

Frontend

Frontend znajduje się w katalogu frontend. Najważniejsze pliki:

- frontend/src/main.js - start aplikacji,
- frontend/src/router/index.js - routing,
- frontend/src/App.vue - kontener aplikacji,
- frontend/src/components/home_page/HomePage.vue - główny panel użytkownika,
- frontend/src/composables/useAuth.js - stan logowania.

Główne komponenty:

- AddedItems.vue - katalog dodanych przedmiotów,
- AddedShopPurchaseLists.vue - koszyki sklepowe,
- PublicPurchasePlans.vue - dofinansowania i plany publiczne,
- PurchaseRequest.vue - wnioski o zamówienie,
- Settlement.vue - rozliczenia i faktury,
- Shops.vue - sklepy.

Przepływ danych

```

Użytkownik
|
v
Vue component
|
| fetch("http://localhost:8080/api/...")
v
FastAPI route
|
v
SQLModel Session
|
v
PostgreSQL

```

W projekcie nie ma oddzielnej warstwy serwisów. Logika biznesowa jest obecnie skupiona w endpointach i funkcjach pomocniczych modułów routes. Modele SQLModel pełnią rolę mapowania encji relacyjnych.

Inicjalizacja bazy

Przy starcie backend:

1. tworzy tabele przez SQLModel.metadata.create_all(engine),
2. wykonuje funkcję migracyjną migrate_project_budgets(),
3. próbuje załadować dane z backend/scripts/mockup_data.sql.

Funkcja migracyjna obsługuje ewolucję projektu: dodaje kolumny, tabele pomocnicze, statusy

faktur i pola używane przez finalizację wniosków.

Konfiguracja runtime

Backend korzysta z wartości DATABASE_URL. W docker-compose.yml zmienna wskazuje na kontener bazy:

```
postgresql://budgetflowfusion:budgetflowfusion@db:5432/budgetflowfusion
```

Porty lokalne:

```
8080 -> FastAPI backend
5431 -> PostgreSQL host port, wewnątrz kontenera 5432
```

Kontener backendu montuje katalog ./backend jako /backend. Dzięki temu zmiany w kodzie backendu są widoczne w kontenerze po restarcie procesu.

Zależności techniczne

Backend:

- python:3.10-slim jako obraz bazowy,
- fastapi[standard] jako framework HTTP,
- sqlmodel jako ORM i deklaracja schematu,
- psycopg2 jako sterownik PostgreSQL,
- uvicorn[standard] jako serwer ASGI,
- pytest jako framework testowy.

Frontend:

- vue w wersji z gałęzi 3.5,
- vue-router do routingu SPA,
- vite jako dev server i bundler,
- @vitejs/plugin-vue do obsługi komponentów .vue.

Rejestracja tras backendu

Plik backend/src/__init__.py tworzy globalną instancję app i na końcu importuje src.routes. Import modułu routes powoduje załadowanie plików tras, które dekorują tę samą instancję app.

```
app = FastAPI(lifespan=lifespan)
app.add_middleware(CORSMiddleware, ...)
import src.routes
```

Konsekwencje:

- endpointy są rejestrowane przez efekt uboczny importu,
- trasy używają wspólnej zależności get_session(),
- testy mogą podmienić zależność get_session przez app.dependency_overrides.

Cykl życia requestu

```

HTTP request
|
v
FastAPI route function
|
+--> Pydantic BaseModel waliduje payload
|
+--> Depends(get_session) otwiera Session(engine)
|
+--> SQLAlchemy select/session.get/session.add
|
+--> session.commit albo rollback przy wyjątku
|
v
Pydantic response_model / dict / SQLAlchemy object

```

Warstwa API zwraca mieszankę:

- modeli SQLAlchemy bezpośrednio, np. Shop albo ShopPurchaseList,
- dedykowanych modeli wyjściowych BaseModel, np. PurchaseRequestOut,
- prostych słowników statusu, np. {"status": "success"}.

Mapowanie frontend -> backend

```

HomePage.vue
  /api/dashboard/budget_summary
  /api/project_budgets
  /api/fundings

PublicPurchasePlans.vue
  /api/fundings
  /api/project_budgets
  /api/public_purchase_plan_lists
  /api/public_purchase_plans

AddedShopPurchaseLists.vue + ShopPurchaseListDetails.vue
  /api/lists
  /api/lists/{id}/items
  /api/lists/{id}/close
  /api/lists/{id}/reopen
  /api/lists/{id}/market_research

PurchaseRequest.vue
  /api/purchase_requests
  /api/create_purchase_requests
  /api/purchase_requests/{id}/prepare_finalization
  /api/purchase_requests/{id}/finalize
  /api/purchase_requests/{id}/send_to_settlement
  /api/purchase_requests/{id}/settlement_lines

Settlement.vue
  /api/settlements
  /api/settlements/history
  /api/invoices
  /api/settlements/{id}/invoices
  /api/settlements/{id}/complete

```

Stan sesji frontendu

Sesja użytkownika nie jest tokenem JWT. useAuth.js zapisuje obiekt użytkownika w localStorage:

```
{
  id,
  association_id,
  firstName,
  lastName,
  email,
  circleName,
  position,
  inSAP,
  projectFinanceManagerId,
  role
}
```

Router sprawdza tylko, czy obiekt istnieje. Przy odświeżeniu strony restoreSession() odtwarza użytkownika z localStorage.

Konsekwencja bezpieczeństwa: aplikacja ma kontrolę dostępu głównie na poziomie interfejsu oraz wybranych walidacji backendowych. Pełna produkcyjna autoryzacja wymagałaby sesji serwerowej albo tokenów i spójnego middleware uprawnień.

Model danych

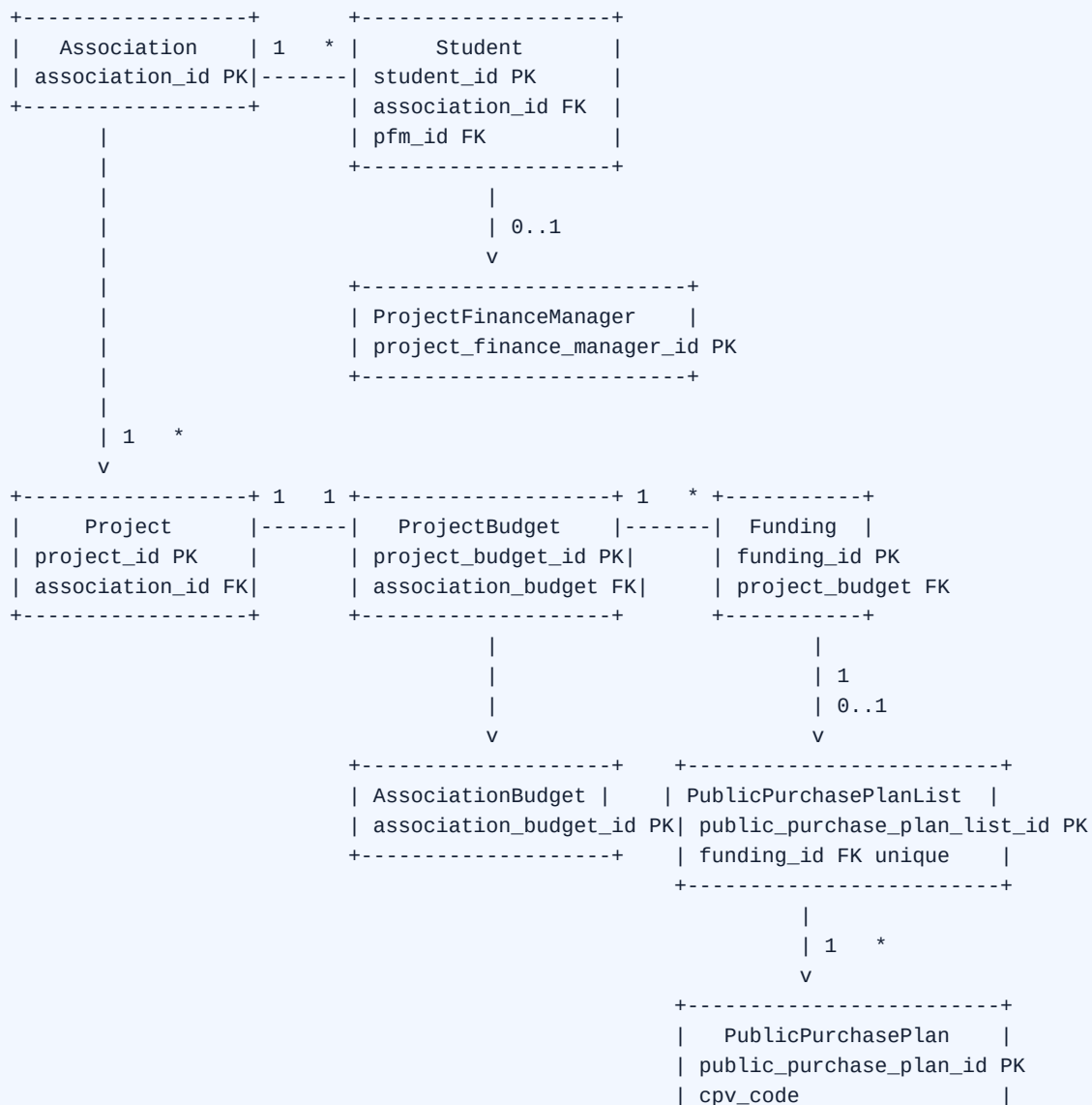
Charakter modelu

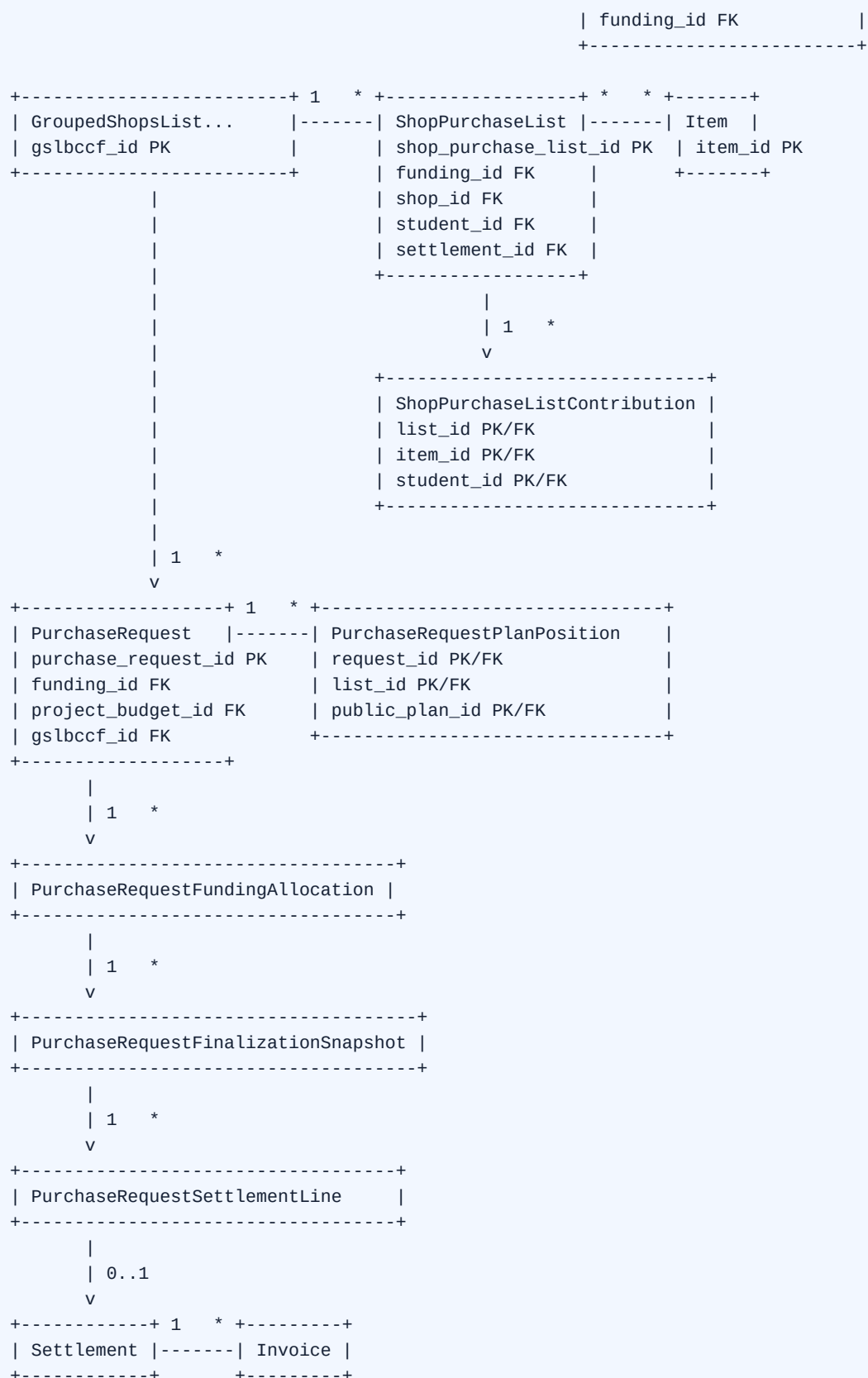
Model danych jest relacyjnym modelem domeny zakupowo-budżetowej koła naukowego. Encje są zdefiniowane w backend/src/relations.py jako klasy SQLAlchemy. W runtime tabele tworzone są przez SQLAlchemy.metadata.create_all(engine), a brakujące kolumny i tabele historyczne uzupełnia funkcja migrate_project_budgets().

Konwencje diagramów:

PK klucz główny
FK klucz obcy
1 dokładnie jeden
0..1 zero albo jeden
* wiele

Diagram logiczny całego modelu





Podmodel użytkowników i organizacji

```

Association
  association_id PK
  association_name

Student
  student_id PK
  name
  surname
  login
  password_hash
  position
  is_in_sap
  association_id FK
  project_finance_manager_id FK nullable

ProjectFinanceManager
  project_finance_manager_id PK
  login
  password_hash
  access

Project
  project_id PK
  project_name
  description
  allocated_budget
  rest_of_budget
  association_id FK

```

Opis:

- Association jest granicą organizacyjną. Po tej encji filtrowane są projekty, użytkownicy, budżety, listy i wnioski.
- Student reprezentuje konto użytkownika. Rola skarbnika wynika z ustawionego `project_finance_manager_id`.
- ProjectFinanceManager jest rekordem uprawniającym do funkcji skarbnika.
- Project reprezentuje sekcję lub projekt koła. Z projektem jest połączony dokładnie jeden ProjectBudget.

Kardynalności:

```

Association 1 -- * Student
Association 1 -- * Project
Project 1 -- 0..1 ProjectBudget
Student 0..1 -- 1 ProjectFinanceManager

```

Podmodel budżetów i dofinansowań

```

AssociationBudget
  association_budget_id PK
  association_budget_name
  total_budget
  spent_money

ProjectBudget
  project_budget_id PK

```

```
project_budget_name
total_budget
spent_money
association_budget_id FK
project_id FK unique
```

Funding

```
funding_id PK
funding_name
organizer
signing_person
spending_deadline
funding_price
spent_money
project_id FK
project_budget_id FK
association_budget_id FK
```

FundingTask

```
funding_task_id PK
funding_id FK
task_name
task_budget
```

Opis:

- AssociationBudget jest budżetem zbiorczym koła.
- ProjectBudget jest budżetem sekcji/projektu.
- Funding jest faktycznym źródłem pieniędzy i najważniejszą encją finansową.
- FundingTask rozbija dofinansowanie na zadania opisowe.

Decyzja projektowa:

```
Budżet sekcji = suma Funding.funding_price dla tej sekcji.
Budżet koła   = suma budżetów sekcji.
```

Dlatego ProjectBudget.total_budget i AssociationBudget.total_budget nie powinny być traktowane jako niezależne kwoty biznesowe. Są zgodne z dofinansowaniami albo pełnią rolę danych bazowych po migracji.

Wzory:

```
project_total_budget =
    SUM(Funding.funding_price WHERE project_budget_id = X)

project_spent_money =
    SUM(Funding.spent_money WHERE project_budget_id = X)

project_reserved =
    SUM(PurchaseRequest.budget_allocated_for_the_order
        WHERE project_budget_id = X)

project_available =
    project_total_budget - project_spent_money - project_reserved
```

Podmodel planów zamówień publicznych

```
Funding
|
| 1 : 0..1
v
PublicPurchasePlanList
  public_purchase_plan_list_id PK
  public_plan_list_name
  plan_year
  plan_number
  fund_responsible_person
  euro_exchange_rate
  funding_id FK unique
|
| 1 : *
v
PublicPurchasePlan
  public_purchase_plan_id PK
  public_purchase_plan_name
  cpv_code
  plan_position_number
  cost
  funding_id FK
  gslbccf_id FK
  public_purchase_plan_list_id FK
```

Opis:

- PublicPurchasePlanList jest nagłówkiem planu dla jednego dofinansowania i roku.
- PublicPurchasePlan jest pozycją planu. W praktyce reprezentuje kod CPV i planowaną kwotę.
- cpv_code jest tekstem, nie liczbą.
- gslbccf_id łączy pozycję planu z grupą koszyków zakupowych.

Reguły:

- jedno dofinansowanie ma najwyżej jeden PublicPurchasePlanList,
- CPV musi być unikalny w ramach jednego planu,
- koszt pozycji planu musi być dodatni,
- pozycji użytej we wniosku nie wolno usuwać,
- pozycja z podpiętym koszykiem nie powinna być usuwana.

Wykorzystanie pozycji planu:

```
used_amount =
  SUM(PurchaseRequestPlanPosition.allocated_amount)
  + legacy SUM(PurchaseRequest.budget_allocated_for_the_order)

remaining_amount =
  PublicPurchasePlan.cost - used_amount
```

Podmodel katalogu i list zakupów

ProductCategory
product_category_id PK
product_category_name
description
cpv
shop_purchase_list_id FK nullable
public_purchase_plan_id FK nullable

ProductSubcategory
product_subcategory_id PK
product_subcategory_name
description
product_category_id FK nullable

Shop
shop_id PK
shop_name
link
opinion
status
created_by_student_id
address
delivery_time
is_recommended
free_delivery_threshold

Item
item_id PK
name
price
currency
link
created_at
tax_rate
status
product_subcategory_id FK
student_id FK
shop_id FK

ShopPurchaseList
shop_purchase_list_id PK
priority
name
cost
created_at
market_research_comment
market_research_file_name
gslbccf_id FK nullable
settlement_id FK nullable
funding_id FK
shop_id FK
student_id FK

ShopPurchaseListItem
shop_purchase_list_id PK/FK
item_id PK/FK
amount

ShopPurchaseListItemContribution
shop_purchase_list_id PK/FK

```
item_id PK/FK
student_id PK/FK
amount
created_at
```

Opis:

- ProductCategory może być powiązana z pozycją planu, dzięki czemu CPV przechodzi z planu do katalogu.
- ProductSubcategory grupuje przedmioty wewnątrz kategorii.
- Item jest wspólnym katalogowym przedmiotem. Aktualnie nowe przedmioty trafiają do katalogu jako approved.
- Shop przechowuje sklep i jego status.
- ShopPurchaseList jest koszykiem jednego sklepu.
- ShopPurchaseListItem zapisuje łączną ilość przedmiotu na liście.
- ShopPurchaseListItemContribution zapisuje wkład konkretnego studenta.

Diagram wkładów studentów:

```
Student Jan dodaje 2 szt.
Student Ola dodaje 3 szt.
    |
    v
ShopPurchaseListItem.amount = 5
    |
    +-- Contribution Jan = 2
    +-- Contribution Ola = 3
```

Dzięki temu usunięcie pozycji przez zwykłego studenta usuwa tylko jego wkład, a nie cały przedmiot z koszyka.

Stan listy zakupów:

```
settlement_id = NULL
    lista otwarta, można dodawać i usuwać pozycje

settlement_id != NULL
    lista zamknięta, nie można zmieniać pozycji
```

Podmodel wniosków

```
PurchaseRequest
purchase_request_id PK
purchase_request_name
budget_allocated_for_the_order
if_service
used_cpv_id
created_at
can_add
document_request_name
contract_value_date
euro_exchange_rate
main_cpv_code
final_net_total
final_gross_total
```

```
finalization_status
finalized_at
project_budget_id FK
funding_id FK
public_purchase_plan_id FK nullable
plan_exception_justification
plan_compliance_status
gslbccf_id FK nullable
project_finance_manager_id FK
```

```
PurchaseRequestFundingAllocation
purchase_request_id PK/FK
funding_id PK/FK
allocated_amount
```

```
PurchaseRequestPlanPosition
purchase_request_id PK/FK
shop_purchase_list_id PK/FK
public_purchase_plan_id PK/FK
allocated_amount
```

Opis:

- PurchaseRequest jest głównym dokumentem zakupowym.
- funding_id jest głównym finansowaniem wniosku.
- wiele finansowań obsługuje PurchaseRequestFundingAllocation.
- public_purchase_plan_id jest główną lub historyczną pozycją planu.
- wiele pozycji planu obsługuje PurchaseRequestPlanPosition.
- gslbccf_id wiąże wniosek z koszykami.

Diagram alokacji:

```
PurchaseRequest "Zakup elektroniki"
|
+-- FundingAllocation: Grant A, 1000 PLN
+-- FundingAllocation: Grant B, 500 PLN
|
+-- PlanPosition: CPV 31700000, koszyk TME, 900 PLN
+-- PlanPosition: CPV 30200000, koszyk X-kom, 600 PLN
```

Status zgodności z planem:

```
draft
  wniosek bez pełnych danych planu albo kwota 0

compliant
  alokacje mieszczą się w pozycjach planu

requires_approval
  alokacja przekracza plan i wymaga uzasadnienia
```

Status finalizacji:

```
draft
|
v
prepared
```



```

|
v
accounting_pending
|
v
settlement
|
v
settled

```

Podmodel finalizacji i rozliczeń

PurchaseRequestFinalizationSnapshot

```

snapshot_id PK
purchase_request_id FK
shop_purchase_list_id FK
public_purchase_plan_id FK
funding_id FK
shop_name
funding_name
plan_name
plan_number
fund_responsible_person
plan_position_number
cpv_code
planned_net_amount
allocated_net_amount
allocated_eur_amount
allocated_gross_amount
is_main_cpv
created_at

```

PurchaseRequestSettlementLine

```

settlement_line_id PK
purchase_request_id FK
shop_purchase_list_id FK
invoice_id FK nullable
shop_name
purchase_description
planned_gross_amount
actual_gross_amount
is_extra
created_at
updated_at

```

Settlement

```

settlement_id PK
created_at
paid_by_project_finance_manager_id FK nullable
purchase_request_id FK nullable

```

Invoice

```

invoice_id PK
number
issue_date
seller_name
seller_nip
net_total

```

```
vat_total
status
created_at
settlement_id FK
project_finance_manager_id FK nullable
```

Opis:

- PurchaseRequestFinalizationSnapshot chroni dokument historyczny przed zmianami w planie i finansowaniu.
- PurchaseRequestSettlementLine jest linią rozliczenia wniosku.
- Settlement grupuje faktury i listy zakupów na etapie rozliczeń.
- Invoice reprezentuje dokument księgowy.

Diagram rozliczenia:

```
PurchaseRequest
|
| send_to_settlement
v
Settlement
|
+-- Invoice FV/01/2026
+-- Invoice FV/02/2026
|
+-- SettlementLine sklep A -> invoice FV/01/2026
+-- SettlementLine sklep B -> invoice FV/02/2026
```

Reguła zakończenia:

Każda PurchaseRequestSettlementLine musi mieć invoice_id.
Dopiero wtedy Settlement może ustawić wniosek jako settled.

Enums i statusy

```
InvoiceStatus
pending
accepted
rejected
paid
returned
arrived
```

```
Currency
PLN
EUR
USD
```

```
ItemStatus
draft
pending
approved
rejected
```

Najważniejsze zależności obliczeniowe

Budżet dofinansowania:

```
available_money =
    Funding.funding_price - Funding.spent_money

purchase_requests_total_allocated =
    SUM(PurchaseRequestFundingAllocation.allocated_amount)

available_after_purchase_requests =
    available_money - purchase_requests_total_allocated
```

Budżet sekcji:

```
total_budget =
    SUM(Funding.funding_price)

spent_money =
    SUM(Funding.spent_money)

reserved =
    SUM(PurchaseRequest.budget_allocated_for_the_order)

available_after_purchase_requests =
    total_budget - spent_money - reserved
```

Budżet koła:

```
total_budget =
    SUM(ProjectBudget.total_budget liczonych z Funding)

spent_money =
    SUM(ProjectBudget.spent_money liczonych z Funding)

reserved =
    SUM(PurchaseRequest.budget_allocated_for_the_order dla projektów koła)

available_after_purchase_requests =
    total_budget - spent_money - reserved
```

Kwoty planu:

```
used_amount =
    SUM(PurchaseRequestPlanPosition.allocated_amount)

remaining_amount =
    PublicPurchasePlan.cost - used_amount
```

Uwagi projektowe

- Model przechowuje część pól historycznych dla zgodności z wcześniejszą wersją projektu, np. `PurchaseRequest.public_purchase_plan_id` obok `PurchaseRequestPlanPosition`.
- `GroupedShopsListByCpvCategoryAndFunding` jest techniczną grupą spinającą plan, wniosek i koszyki.

- settlement_id na liście zakupów jest praktycznym znacznikiem zamknięcia listy.
- used_cpv_id jest tekstem, bo CPV może zawierać zera wiodące albo myślnik.
- Finalizacja pracuje na kwotach netto dla planu i brutto dla koszyków, dlatego snapshot zapisuje oba warianty kwot.

Procesy biznesowe

Proces 1: Dodanie przedmiotu do katalogu

```
Użytkownik
|
v
Formularz dodania przedmiotu
|
v
Backend sprawdza autora
|
+--> zapis Item(status = approved)
|
v
Przedmiot trafia do wspólnego katalogu
```

Opis:

1. Użytkownik wybiera kategorię, podkategorię, sklep i podaje dane przedmiotu.
2. System zapisuje przedmiot.
3. Aktualny endpoint POST /api/items zapisuje status="approved".
4. GET /api/items zwraca tylko przedmioty zaakceptowane, więc nowy przedmiot jest od razu widoczny w katalogu.

Proces 2: Utworzenie dofinansowania

```
Skarbnik
|
v
Wybór sekcji/projektu
|
v
Dane dofinansowania
|
v
Zadania budżetowe
|
v
Funding + FundingTask
```

Opis:

1. Skarbnik podaje nazwę dofinansowania, organizatora, osobę podpisującą, termin wydatkowania i kwotę.
2. Dofinansowanie jest przypisywane do budżetu sekcji.
3. Opcjonalnie dodawane są zadania budżetowe.
4. Suma zadań nie może przekroczyć kwoty dofinansowania.

Proces 3: Utworzenie planu zamówień publicznych

```
Dofinansowanie
|
v
PublicPurchasePlanList
|
v
Pozycje CPV
|
v
PublicPurchasePlan
```

Opis:

1. Skarbnik wybiera dofinansowanie.
2. Tworzy roczny plan dla tego dofinansowania.
3. Uzupełnia numer planu, kurs euro i osobę odpowiedzialną.
4. Dodaje pozycje planu: CPV, numer pozycji, opis i kwotę.
5. Pozycje planu mogą zostać powiązane z kategoriami produktowymi.

Proces 4: Tworzenie koszyka sklepowego

```
Skarbnik lub członek
|
v
Wybór wniosku / pozycji planu
|
v
Wybór sklepu i dofinansowania
|
v
ShopPurchaseList
|
v
Dodawanie pozycji z katalogu
```

Opis:

1. Koszyk sklepowy jest tworzony dla konkretnego sklepu.
2. Koszyk posiada dofinansowanie oraz kontekst grupy zakupowej.
3. Jeżeli koszyk jest tworzony przez zwykłego członka, musi być powiązany z istniejącym otwartym zamówieniem/wnioskiem.
4. Do koszyka dodawane są przedmioty z katalogu.
5. System zapisuje wkład poszczególnych studentów.

Proces 5: Złożenie wniosku o zamówienie

```
Skarbnik
|
v
Dane wniosku
|
v
Alokacje dofinansowań
|
```

```

    v
Pozycje planu publicznego
|
v
walidacja budżetu i planu
|
+--> zgodny z planem
|
+--> wymaga zgody i uzasadnienia
|
v
PurchaseRequest

```

Opis:

1. Skarbnik tworzy wniosek.
2. Wniosek otrzymuje kwotę, typ zakupu i kontekst finansowy.
3. Możliwe jest wskazanie wielu alokacji dofinansowań.
4. Możliwe jest wskazanie wielu pozycji planu publicznego.
5. System sprawdza dostępne środki i wykorzystanie pozycji planu.
6. Jeżeli plan jest przekroczony, wymagane jest uzasadnienie.

Proces 6: Finalizacja wniosku

```

PurchaseRequest
|
v
prepare_finalization
|
v
sprawdzenie koszyków i kwot
|
v
finalize
|
v
snapshot + linie rozliczeń
|
v
accounting_pending

```

Opis:

1. System sprawdza, czy wniosek może być finalizowany.
2. Koszyki zostają zamknięte dla dalszych zmian.
3. System liczy wartości netto, brutto i EUR.
4. Zapisywany jest snapshot danych planu.
5. Tworzone są linie rozliczeniowe.
6. Wniosek przechodzi do statusu oczekiwania na księgowość.

Proces 7: Rozliczenie i faktury

```

Wniosek po finalizacji
|
v
send_to_settlement
|
v
Settlement
|
v
Invoice + SettlementLine
|
v
complete
|
v
settled

```

Opis:

1. Wniosek trafia do rozliczeń.
2. Skarbnik lub osoba rozliczająca dodaje faktury.
3. Linie rozliczenia są przypisywane do faktur i kwot rzeczywistych.
4. System pokazuje różnice między planem a faktycznym wydatkiem.
5. Po zakończeniu rozliczenia wniosek może trafić do historii.

Proces 8: Kontrola planu publicznego

```

PublicPurchasePlan.cost
|
v
minus suma PurchaseRequestPlanPosition.allocated_amount
|
v
remaining_amount

```

Jeżeli nowa alokacja przekracza pozostałą kwotę planu, wniosek otrzymuje status `requires_approval`. System nie blokuje bezwzględnie takiego wniosku, ponieważ w praktyce możliwe jest uzasadnione odstępstwo, ale wymaga opisanie powodu.

Proces techniczny: wniosek z zamkniętego koszyka

Ten wariant odpowiada obecnemu wymaganiu, że skarbnik może utworzyć wniosek ze swoich zamkniętych list zakupów.

```

ShopPurchaseList(settlement_id = NULL)
|
| PATCH /api/lists/{id}/close
v
Settlement(purchase_request_id = NULL)
ShopPurchaseList(settlement_id = settlement.id)
|
| GET /api/lists/closed_for_purchase_requests?project_finance_manager_id=X
v
lista zamkniętych koszyków skarbnika
|
| POST /api/create_purchase_requests

```



```
|   shop_purchase_list_id = id
v
PurchaseRequest
Settlement(purchase_request_id = purchase_request.id)
```

Walidacje:

- koszyk musi istnieć,
- koszyk musi mieć settlement_id, czyli być zamknięty,
- koszyk musi być utworzony przez studenta powiązanego z tym samym project_finance_manager_id,
- settlement koszyka nie może być już powiązany z innym wnioskiem,
- dofinansowanie koszyka wyznacza funding_id i project_budget_id wniosku,
- kwota wniosku jest liczona z pozycji koszyka przez _shop_purchase_list_total.

Proces techniczny: widoczność list zakupów

Endpoint GET /api/lists obsługuje kilka trybów filtrowania.

```
treasurer_view=true + association_id
-> listy studentów z danego koła

student_id
-> listy konkretnego studenta

association_id
-> listy studentów z danego koła

open_only=true
-> tylko listy z settlement_id = NULL

purchase_request_id
-> listy z gslbccf_id wniosku

public_purchase_plan_id
-> listy z gslbccf_id pozycji planu
```

Praktyczne reguły wynikające z frontendowego użycia:

- skarbnik widzi swoje listy oraz listy skarbników z tego samego koła,
- zwykły student widzi otwarte listy powiązane z jego kołem i kontekstem zamówienia,
- zamykać listę może tylko skarbnik będący jej autorem,
- zwykły student może usuwać z listy tylko ilości zapisane w swoim ShopPurchaseListItemContribution.

Proces techniczny: kontrola budżetu przy wniosku

POST /api/create_purchase_requests wykonuje kontrolę w tej kolejności:

1. Ustala źródło danych: payload, alokacje finansowania albo zamknięty koszyk.
2. Ustala project_budget_id i funding_id.
3. Sprawdza, czy dofinansowanie należy do wskazanego budżetu sekcji.
4. Liczy dostępny budżet sekcji.
5. Liczy dostępne środki dofinansowania.

6. Odrzuca wniosek, jeśli kwota przekracza budżet sekcji.
7. Odrzuca wniosek, jeśli kwota przekracza dofinansowanie.
8. Sprawdza pozycje planu publicznego i ich finansowanie.
9. Nadaje plan_compliance_status.
10. Tworzy lub reuse'uje gslbccf_id dla grupy zakupowej.
11. Zapisuje PurchaseRequest.
12. Zapisuje PurchaseRequestFundingAllocation.
13. Zapisuje PurchaseRequestPlanPosition.

Status zgodności z planem:

```
budget_allocated <= 0
-> draft

brak pozycji planu
-> draft

suma alokacji planu <= pozostała kwota pozycji
-> compliant

suma alokacji planu > pozostała kwota pozycji
-> requires_approval

requires_approval bez uzasadnienia
-> HTTP 400
```

Proces techniczny: finalizacja

Finalizacja składa się z trzech endpointów:

```
POST /api/purchase_requests/{id}/prepare_finalization
PATCH /api/purchase_requests/{id}/finalization_draft
POST /api/purchase_requests/{id}/finalize
```

prepare_finalization:

- wymaga koszyków w grupie gslbccf_id,
- wymaga pozycji PurchaseRequestPlanPosition,
- tworzy lub aktualizuje Settlement dla każdego koszyka,
- przelicza koszt koszyka z pozycji ShopPurchaseListItem,
- ustawia can_add = False i finalization_status = prepared.

finalization_draft:

- działa tylko dla statusów prepared i draft,
- zapisuje document_request_name, euro_exchange_rate i contract_value_date,
- wymaga dodatniego kursu euro, jeśli kurs został podany.

finalize:

- wymaga niepustej nazwy dokumentu,
- wymaga dodatniego kursu euro,
- tworzy PurchaseRequestFinalizationSnapshot,
- wylicza główny CPV jako CPV z największą kwotą netto,
- przepisuje used_cpv_id na główny CPV,

- ustawia final_net_total i final_gross_total,
- aktualizuje kwotę wniosku do wartości brutto,
- ustawia finalization_status = accounting_pending.

Proces techniczny: rozliczenie

```
accounting_pending
|
| POST /api/purchase_requests/{id}/send_to_settlement
v
settlement
|
| faktury + przypisanie invoice_id do linii
v
POST /api/settlements/{id}/complete
|
v
settled
```

Warunki zakończenia:

- settlement musi być powiązany z wnioskiem,
- wniosek musi istnieć,
- wszystkie PurchaseRequestSettlementLine muszą mieć invoice_id,
- po sukcesie backend ustawia PurchaseRequest.finalization_status na settled.

API backendu

Konwencje

Backend udostępnia REST API pod adresem:

```
http://localhost:8080/api
```

Dane są przesyłane jako JSON. Frontend komunikuje się z backendem przez funkcję fetch.

Autoryzacja w obecnym kształcie projektu jest uproszczona. Stan użytkownika jest przechowywany po stronie frontendu w localStorage, a wiele endpointów przyjmuje identyfikatory użytkownika, skarbnika lub koła jako parametry.

Logowanie i rejestracja

POST /api/login Loguje użytkownika po e-mailu i hasle.

POST /api/register Rejestruje użytkownika. Może utworzyć zwykłego członka albo skarbnika.

Członkowie

GET /api/students Zwraca listę studentów/członków.

Katalog przedmiotów

GET /api/items Pobiera katalog przedmiotów.

GET /api/items/grouped Pobiera przedmioty pogrupowane do wygodnego użycia w interfejsie.

GET /api/items/pending Pobiera przedmioty oczekujące na akceptację.

POST /api/items Dodaje nowy przedmiot.

PATCH /api/items/{item_id}/approve Akceptuje przedmiot.

DELETE /api/items/{item_id}/reject Odrzuca i usuwa przedmiot.

Kategorie

GET /api/categories Lista kategorii.

POST /api/categories Dodanie kategorii wraz z CPV.

GET /api/subcategories Lista podkategorii.

GET /api/subcategories/pending Podkategorie oczekujące.

POST /api/subcategories Dodanie podkategorii.

PATCH /api/subcategories/{subcategory_id}/assign-category Przypisanie podkategorii do kategorii.

Sklepy

GET /api/shops Pobiera sklepy. Może uwzględniać oczekujące sklepy.

POST /api/shops Dodaje sklep.

PATCH /api/shops/{shop_id} Edytuje sklep.

PATCH /api/shops/{shop_id}/approve Akceptuje sklep.

DELETE /api/shops/{shop_id}/reject Odrzuca sklep.

Budżety i dofinansowania

GET /api/association_budgets Pobiera budżety koła. Przyjmuje opcjonalnie association_id.

GET /api/project_budgets Pobiera budżety projektów/sekcji.

GET /api/dashboard/budget_summary Zwraca agregat budżetowy dla pulpitu.

GET /api/fundings Pobiera dofinansowania, opcjonalnie filtrowane po kole.

POST /api/fundings Tworzy dofinansowanie i zadania budżetowe.

PATCH /api/fundings/{funding_id} Aktualizuje dofinansowanie.

Plany publiczne

POST /api/public_purchase_plan_lists Tworzy albo aktualizuje roczny plan publiczny dla dofinansowania.

GET /api/public_purchase_plan_lists Pobiera plany publiczne, opcjonalnie po association_id albo funding_id.

GET /api/public_purchase_plan_lists/{id} Pobiera szczegóły planu.

POST /api/public_purchase_plans Dodaje pozycję CPV do planu.

PATCH /api/public_purchase_plans/{id} Edytuje pozycję planu.

DELETE /api/public_purchase_plans/{id} Usuwa pozycję planu, jeżeli nie została użyta w koszykach lub wnioskach.

Listy zakupów

GET /api/lists Pobiera koszyki sklepowe. Obsługuje filtry: użytkownik, koło, sklep, plan publiczny, wniosek, grupa oraz tylko otwarte.

POST /api/lists Tworzy koszyk sklepowy.

PATCH /api/lists/{list_id}/close Zamyka koszyk i tworzy/wiąże rozliczenie.

PATCH /api/lists/{list_id}/reopen Otwiera ponownie koszyk.

GET /api/lists/closed_for_purchase_requests Pobiera zamknięte koszyki, z których można utworzyć wniosek.

GET /api/lists/{list_id} Szczegóły koszyka.

PATCH /api/lists/{list_id}/market_research Zapisuje informację o rozeznaniu rynku.

DELETE /api/lists/{list_id} Usuwa koszyk.

GET /api/lists/{list_id}/items Pobiera przedmioty w koszyku.

POST /api/lists/{list_id}/items Dodaje przedmiot do koszyka.

DELETE /api/lists/{list_id}/items/{item_id} Usuwa przedmiot z koszyka.

Wnioski

GET /api/purchase_requests Pobiera wnioski. Może filtrować po kole.

GET /api/purchase_requests/detail/{purchase_request_id} Szczegóły wniosku.

GET /api/purchase_requests/{project_finance_manager_id} Wnioski konkretnego skarbnika.

POST /api/create_purchase_requests Tworzy wniosek.

PATCH /api/purchase_requests/{purchase_request_id} Aktualizuje wniosek.

DELETE /api/purchase_requests/{purchase_request_id} Usuwa wniosek.

Finalizacja wniosku

POST /api/purchase_requests/{id}/prepare_finalization Przygotowuje wniosek do finalizacji, zamyka dodawanie i tworzy potrzebne rozliczenia.

PATCH /api/purchase_requests/{id}/finalization_draft Zapisuje robocze dane dokumentowe finalizacji.

POST /api/purchase_requests/{id}/finalize Finalizuje wniosek, zapisuje snapshot, wylicza wartości i ustawia status oczekiwania na księgowość.

POST /api/purchase_requests/{id}/send_to_settlement Przekazuje wniosek do rozliczeń.

POST /api/purchase_requests/{id}/return_to_open Cofa przygotowany wniosek do otwartego stanu.

Linie rozliczeniowe wniosku

GET /api/purchase_requests/{id}/settlement_lines Pobiera linie rozliczenia.

PUT /api/purchase_requests/{id}/settlement_lines Zapisuje komplet linii rozliczenia.

PATCH /api/purchase_request_settlement_lines/{id} Aktualizuje pojedynczą linię.

POST /api/purchase_requests/{id}/settlement_lines/extra Dodaje dodatkową pozycję rozliczenia po przekazaniu do rozliczeń.

Rozliczenia i faktury

GET /api/settlements Pobiera rozliczenia.

GET /api/settlements/history Pobiera historię zakończonych rozliczeń.

GET /api/settlements/manager/{project_finance_manager_id} Rozliczenia konkretnego skarbnika.

POST /api/settlements Tworzy rozliczenie.

POST /api/settlements/{settlement_id}/invoices Dodaje fakturę do rozliczenia.

PATCH /api/invoices/{invoice_id} Edytuje fakturę.

GET /api/invoices Pobiera faktury z filtrami statusu, skarbnika, wniosku i koła.

POST /api/settlements/{settlement_id}/complete Kończy rozliczenie.

PATCH /api/settlements/{settlement_id}/status Zmienia status rozliczenia.

Kontrakty danych: logowanie i użytkownik

POST /api/login przyjmuje:

```
{
  "email": "student@example.com",
  "password": "haslo"
}
```

Odpowiedź sukcesu:

```
{
  "id": 1,
  "firstName": "Jan",
  "lastName": "Kowalski",
  "email": "student@example.com",
  "circleName": "Koło Robotyki",
  "association_id": 1,
  "position": "member",
  "inSAP": true,
  "project_finance_manager_id": null,
  "role": "member"
}
```

Jeżeli użytkownik ma project_finance_manager_id, role ma wartość treasurer.

POST /api/register przyjmuje:

```
{
  "name": "Jan",
  "surname": "Kowalski",
  "login": "student@example.com",
  "password": "haslo",
  "position": "Sekcja mechaniczna",
  "is_in_sap": true,
  "association_name": "Koło Robotyki",
  "is_treasurer": false
}
```

Dla is_treasurer=true backend tworzy dodatkowo rekord ProjectFinanceManager.

Kontrakty danych: katalog i kategorie

POST /api/items:

```
{
  "name": "Silnik krokowy",
  "link": "https://example.com/silnik",
  "price": 120.0,
  "currency": "PLN",
  "product_subcategory_id": 1,
  "student_id": 5,
  "tax_rate": 23
}
```

Reguły:

- musi istnieć co najmniej jeden sklep, bo endpoint przypisuje pierwszy sklep z bazy,
- student_id musi wskazywać istniejącego studenta,
- zapisany status to approved,
- GET /api/items zwraca wyłącznie approved.

POST /api/categories:

```
{
  "name": "Elektronika",
  "cpv": "31700000",
  "student_id": 2
}
```

Reguły:

- student_id jest wymagany,
- student musi mieć project_finance_manager_id,
- brak uprawnień kończy się 403.

POST /api/subcategories:

```
{
  "name": "Mikrokontrolery",
  "product_category_id": 1,
  "student_id": 2
}
```

Jeżeli product_category_id jest podane, backend wymaga skarbnika. Jeżeli jest puste, powstaje podkategoria oczekująca na przypisanie.

Kontrakty danych: dofinansowanie

POST /api/fundings oraz PATCH /api/fundings/{id}:

```
{
  "funding_name": "Grant 2026",
  "organizer": "Politechnika",
  "signing_person": "Anna Testowa",
  "spending_deadline": "2026-12-31",
  "funding_price": 10000.0,
  "project_budget_id": 1,
  "tasks": [
    {
      "task_name": "Części elektroniczne",
      "task_budget": 4000.0
    }
  ]
}
```

Walidacje:

- nazwa, organizator i osoba podpisująca nie mogą być puste,
- funding_price musi być dodatnie,
- project_budget_id musi istnieć,
- każde zadanie ma dodatni budżet,
- suma task_budget nie może przekroczyć funding_price.

Odpowiedź zawiera pola obliczeniowe:

```
available_money = funding_price - spent_money
purchase_requests_total_allocated = suma rezerwacji z wniosków
available_after_purchase_requests = available_money - rezerwacje
```


Kontrakty danych: plany publiczne

POST /api/public_purchase_plan_lists:

```
{
  "funding_id": 1,
  "public_plan_list_name": "Plan ZP 2026",
  "plan_year": 2026,
  "plan_number": "ZP/2026/01",
  "fund_responsible_person": "Anna Testowa",
  "euro_exchange_rate": 4.7
}
```

Reguły:

- funding_id musi istnieć,
- euro_exchange_rate musi być dodatni, jeśli został podany,
- plan_year musi mieścić się w zakresie 2000..2100,
- dla tego samego funding_id endpoint aktualizuje istniejący plan.

POST /api/public_purchase_plans:

```
{
  "public_purchase_plan_list_id": 1,
  "cpv_code": "31700000",
  "plan_position_number": "1.1",
  "cost": 2500.0,
  "description": "Podzespoły elektroniczne",
  "product_category_id": 3
}
```

Reguły:

- cost musi być większe od zera,
- cpv_code nie może być pusty,
- CPV musi być unikalny w ramach jednego planu dofinansowania,
- przy tworzeniu pozycji powstaje grupa GroupedShopsListByCpvCategoryAndFunding,
- jeśli podano product_category_id, kategoria zostaje powiązana z pozycją planu, a jej cpv zostaje ustawione na CPV pozycji.

Odpowiedź pozycji planu zawiera:

used_amount	suma wykorzystania przez wnioski
remaining_amount	cost - used_amount
gslbccf_id	grupa koszyków dla pozycji planu

Kontrakty danych: listy zakupów

GET /api/lists obsługuje filtry:

```
student_id
association_id
shop_id
public_purchase_plan_id
purchase_request_id
```

```
gslbccf_id
open_only
treasurer_view
```

POST /api/lists:

```
{
  "name": "TME - elektronika",
  "priority": 1,
  "cost": 0.0,
  "created_at": "2026-06-18T10:00:00",
  "funding_id": 1,
  "shop_id": 1,
  "student_id": 2,
  "public_purchase_plan_id": 5,
  "purchase_request_id": null,
  "gslbccf_id": null
}
```

Reguły:

- student i dofinansowanie muszą istnieć,
- dofinansowanie musi należeć do koła studenta,
- zwykły użytkownik musi wskazać wniosek albo pozycję planu,
- skarbnik może utworzyć listę bez kontekstu wniosku,
- dla jednej grupy gslbccf_id nie można utworzyć drugiego koszyka tego samego sklepu,
- wybrane dofinansowanie musi mieć środki większe od zera.

POST /api/lists/{list_id}/items:

```
{
  "item_id": 10,
  "amount": 2,
  "student_id": 5
}
```

albo utworzenie nowego przedmiotu przy dodaniu do listy:

```
{
  "name": "Czujnik",
  "link": "https://example.com",
  "price": 50.0,
  "currency": "PLN",
  "product_subcategory_id": 1,
  "tax_rate": 23,
  "amount": 3,
  "student_id": 5
}
```

Reguły:

- lista musi być otwarta,
- amount musi być dodatnie,
- nowy przedmiot wymaga nazwy, ceny i podkategorii z kategorią,
- backend zwiększa ShopPurchaseListItem.amount i zapisuje wkład w

ShopPurchaseListItemContribution.

DELETE /api/lists/{list_id}/items/{item_id}?student_id=5:

- lista musi być otwarta,
- skarbnik będący właścicielem listy może usunąć całą pozycję,
- zwykły student usuwa tylko własny wkład,
- jeśli po odjęciu wkładu ilość spada do zera, usuwany jest cały ShopPurchaseListItem.

Kontrakty danych: wniosek

POST /api/create_purchase_requests:

```
{
  "purchase_request_name": "Zakup elektroniki",
  "budget_allocated_for_the_order": 1200.0,
  "if_service": false,
  "used_cpv_id": "31700000",
  "project_budget_id": 1,
  "funding_id": 1,
  "created_at": "2026-06-18T10:00:00",
  "created_by_user_id": 2,
  "can_add": true,
  "project_finance_manager_id": 1,
  "shop_purchase_list_id": null,
  "public_purchase_plan_id": null,
  "plan_exception_justification": null,
  "funding_allocations": [
    {
      "funding_id": 1,
      "allocated_amount": 1200.0
    }
  ],
  "plan_positions": [
    {
      "shop_purchase_list_id": 3,
      "public_purchase_plan_id": 5,
      "allocated_amount": 1200.0
    }
  ]
}
```

Najważniejsze walidacje:

- project_finance_manager_id jest wymagane,
- jeśli shop_purchase_list_id jest podane, koszyk musi być zamknięty i należeć do tego skarbnika,
- funding_id musi należeć do project_budget_id,
- kwota nie może przekroczyć dostępnego budżetu sekcji,
- kwota nie może przekroczyć dostępnego dofinansowania,
- każda pozycja planu musi należeć do jednego z dofinansowań użytych we wniosku,
- przekroczenie pozycji planu wymaga plan_exception_justification.

Odpowiedź PurchaseRequestOut zawiera między innymi:

```
plan_compliance_status
budget_info
source_shop_purchase_list
funding_allocations
plan_position
plan_positions
finalization_status
```

Kontrakty danych: finalizacja

POST /api/purchase_requests/{id}/prepare_finalization nie przyjmuje payloadu. Odpowiedź PurchaseRequestFinalizationOut zawiera:

```
net_total
gross_total
main_cpv_code
cpv_rows
plan_rows
funding_gross_rows
snapshot_rows
```

PATCH /api/purchase_requests/{id}/finalization_draft:

```
{
  "document_request_name": "Wniosek ZP/12/2026",
  "euro_exchange_rate": 4.7,
  "contract_value_date": "2026-06-18"
}
```

POST /api/purchase_requests/{id}/finalize wymaga tego samego zestawu danych, ale wszystkie pola są formalnie wymagane. Po sukcesie backend ustawia:

```
can_add = false
finalization_status = accounting_pending
finalized_at = now()
used_cpv_id = main_cpv_code
budget_allocated_for_the_order = gross_total
```

Kontrakty danych: rozliczenia i faktury

POST /api/settlements/{settlement_id}/invoices:

```
{
  "number": "FV/01/2026",
  "issue_date": "2026-06-18",
  "seller_name": "Sklep testowy",
  "seller_nip": "1234567890",
  "net_total": 1000.0,
  "vat_total": 230.0,
  "status": "pending",
  "project_finance_manager_id": 1
}
```

Status faktury jest enumem:

```
pending
accepted
rejected
paid
returned
arrived
```

PATCH /api/purchase_request_settlement_lines/{line_id} pozwala ustawić invoice_id i actual_gross_amount. actual_gross_amount nie może być ujemne.

POST /api/settlements/{settlement_id}/complete:

- wymaga powiązania settlementu z wnioskiem,
- wymaga faktury na każdej linii rozliczeniowej,
- ustawia PurchaseRequest.finalization_status = settled.

Dokumentacja użytkownika

Przeznaczenie aplikacji

BudgetFlowFusion służy do prowadzenia zakupów w kole naukowym. Aplikacja pomaga zebrać propozycje przedmiotów, utworzyć listy zakupów, kontrolować budżet, przypisać zakup do dofinansowania i planu zamówień publicznych, a następnie przejść przez wniosek, finalizację, rozliczenie i faktury.

Najważniejsza zasada pracy:

Najpierw środki i plan.
Potem listy zakupów.
Potem wniosek.
Na końcu finalizacja, rozliczenie i faktury.

Role w aplikacji

Zwykły członek koła

Zwykły członek może:

- zalogować się do aplikacji,
- przeglądać wspólny katalog przedmiotów,
- dodawać przedmioty do katalogu,
- przeglądać dostępne listy zakupów,
- dodawać przedmioty do otwartych list,
- usuwać z list tylko te ilości, które sam dodał,
- przeglądać podstawowe informacje o swoim profilu.

Zwykły członek nie prowadzi budżetów, planów publicznych, wniosków ani rozliczeń.

Skarbnik

Skarbnik może:

- zarządzać dofinansowaniami,
- prowadzić plany zamówień publicznych,
- tworzyć i zamykać własne listy zakupów,
- widzieć listy skarbników z tego samego koła,
- tworzyć wnioski o zamówienie,
- przypisywać wniosek do dofinansowania i pozycji planu,
- finalizować wniosek,
- przekazywać wniosek do rozliczeń,
- dodawać faktury,
- kończyć rozliczenia,
- obserwować aktualny budżet koła i sekcji.

Logowanie i rejestracja

Logowanie:

1. Otwórz aplikację.

2. Wpisz e-mail i hasło.
3. Po poprawnym logowaniu aplikacja przenosi do panelu głównego.

Rejestracja:

1. Przejdź do formularza rejestracji.
2. Podaj imię, nazwisko, e-mail, hasło, koło naukowe, sekcję i status SAP.
3. Wybierz rolę użytkownika.
4. Po rejestracji zaloguj się na utworzone konto.

Status SAP:

- Jestem w SAP oznacza, że użytkownik figuruje w systemie SAP.
- Nie jestem w SAP oznacza brak takiej informacji w profilu.
- W obecnej wersji status jest informacyjny i nie uruchamia integracji z SAP.

Pulpit

Po zalogowaniu użytkownik widzi swoje dane:

- rolę,
- e-mail,
- koło naukowe,
- sekcję,
- status SAP.

Skarbnik widzi dodatkowo przegląd budżetu:

- budżet całkowity koła,
- kwotę wydaną,
- kwotę zarezerwowaną we wnioskach,
- kwotę dostępną po rezerwacjach,
- procent wykorzystania budżetu.

Kwoty na pulpicie nie są wpisane na sztywno. Są liczone z dofinansowań, wydanych środków i złożonych wniosków. Po utworzeniu wniosku budżet dostępny powinien się zmniejszyć.

Katalog przedmiotów

Zakładka Dodane przedmioty jest wspólnym katalogiem produktów. Katalog jest wspólny dla użytkowników, a nie prywatny dla jednej osoby.

Dodawanie przedmiotu:

1. Otwórz zakładkę Dodane przedmioty.
2. Wybierz dodanie nowego przedmiotu.
3. Podaj nazwę, link, cenę, walutę i podkategorię.
4. Zapisz formularz.
5. Przedmiot pojawi się w katalogu.

Przedmiot w katalogu można później wykorzystać przy dodawaniu pozycji do listy zakupów.

Sklepy

Zakładka Sklepy służy do prowadzenia listy sklepów, z których koło może kupować przedmioty.

Dane sklepu:

- nazwa,
- link,
- opinia,
- próg darmowej dostawy,
- status.

Zwykły użytkownik może zgłosić sklep. Skarbnik może sklep zaakceptować, edytować albo odrzucić, jeśli sklep nie został jeszcze zaakceptowany.

Listy zakupów

Lista zakupów jest koszykiem dla jednego sklepu. Lista może być otwarta albo zamknięta.

Lista otwarta
można dodawać i usuwać pozycje

Lista zamknięta
nie można zmieniać pozycji
może posłużyć do utworzenia wniosku

Widok zwykłego członka:

- widzi dostępne otwarte listy zakupów z jego koła,
- może dodać przedmiot do listy,
- może usunąć tylko własny wkład z listy,
- nie może zamknąć listy.

Widok skarbnika:

- widzi własne listy,
- widzi listy skarbników z tego samego koła,
- może tworzyć nowe listy,
- może zamykać listy utworzone przez siebie,
- może ponownie otworzyć listę, jeśli proces jeszcze na to pozwala.

Dodanie przedmiotu do listy:

1. Otwórz listę zakupów.
2. Wybierz przedmiot z katalogu albo dodaj nowy.
3. Podaj ilość.
4. Zapisz.

Jeśli kilka osób doda ten sam przedmiot, aplikacja pokazuje łączną ilość, ale pamięta wkład każdej osoby oddzielnie.

Usunięcie przedmiotu przez zwykłego członka:

1. Otwórz listę.
2. Wybierz pozycję, którą dodałeś.
3. Usuń swój wkład.

Jeżeli inni użytkownicy też dodali ten sam przedmiot, ich ilości pozostają na liście.

Dofinansowania

Dofinansowanie jest źródłem pieniędzy. Każde dofinansowanie należy do jednej sekcji/projektu.

Tworzenie dofinansowania przez skarbnika:

1. Otwórz zakładkę Plany publiczne.
2. Wybierz sekcję/projekt.
3. Podaj nazwę dofinansowania.
4. Podaj organizatora.
5. Podaj osobę podpisującą.
6. Podaj termin wydatkowania.
7. Podaj kwotę dofinansowania.
8. Opcjonalnie dodaj zadania budżetowe.
9. Zapisz.

Zadania budżetowe pomagają opisać, na co mają zostać przeznaczone środki. Suma zadań nie może być większa niż kwota dofinansowania.

Plany zamówień publicznych

Plan zamówień publicznych jest tworzony dla konkretnego dofinansowania. Plan składa się z pozycji CPV i kwot planowanych.

Utworzenie planu:

1. Otwórz zakładkę Plany publiczne.
2. Wybierz dofinansowanie.
3. Utwórz plan dla tego dofinansowania.
4. Podaj nazwę planu.
5. Podaj rok planu.
6. Podaj numer planu.
7. Podaj osobę odpowiedzialną.
8. Podaj kurs euro, jeśli jest potrzebny do finalizacji.
9. Zapisz.

Dodanie pozycji planu:

1. Otwórz plan dofinansowania.
2. Dodaj pozycję.
3. Podaj kod CPV.
4. Podaj numer pozycji.
5. Podaj opis pozycji.
6. Podaj planowaną kwotę.
7. Opcjonalnie przypisz kategorię produktu.
8. Zapisz.

Znaczenie kwot:

Planowana kwota
ile przewidziano w planie na dany CPV

Wykorzystano
ile przypisano do tej pozycji we wnioskach

Pozostało
planowana kwota minus wykorzystanie

Jeśli wniosek przekracza pozycję planu, trzeba podać uzasadnienie odstępstwa.

Wnioski o zamówienie

Wniosek o zamówienie jest formalnym etapem zakupu. Skarbnik tworzy wniosek wtedy, gdy chce realnie kupić przedmioty z list zakupów.

Utworzenie wniosku z zamkniętej listy:

1. Utwórz listę zakupów.
2. Dodaj do niej przedmioty.
3. Zamknij listę.
4. Przejdź do zakładki Wnioski zamówień.
5. Wybierz zamkniętą listę.
6. Uzupełnij dane wniosku.
7. Wskaż dofinansowanie.
8. Wskaż pozycję planu publicznego.
9. Zapisz wniosek.

Utworzenie wniosku ręcznie:

1. Otwórz zakładkę Wnioski zamówień.
2. Wybierz nowy wniosek.
3. Podaj nazwę wniosku.
4. Podaj kwotę.
5. Wybierz dofinansowanie.
6. Wybierz pozycję lub pozycje planu CPV.
7. Jeżeli kwota przekracza plan, wpisz uzasadnienie.
8. Zapisz.

Status zgodności z planem:

Szkic
wniosek nie ma pełnych danych planu albo kwoty

Zgodny z planem
kwota mieści się w dostępnej pozycji CPV

Wymaga zgody
kwota przekracza plan albo zakup nie był zaplanowany

Finalizacja wniosku

Finalizacja zamyka część zakupową wniosku. Po finalizacji dane są traktowane jako podstawa do rozliczenia.

Przebieg:

1. Otwórz wniosek.
2. Wybierz przygotowanie finalizacji.
3. Sprawdź listy zakupów, CPV, kwoty i dofinansowania.
4. Podaj nazwę dokumentu.
5. Podaj datę wartości umowy.
6. Podaj kurs euro.

7. Zapisz finalizację.

Po finalizacji:

- nie można już dodawać pozycji do list powiązanych z wnioskiem,
- system zapisuje snapshot danych planu,
- system wylicza główny CPV,
- wniosek przechodzi do stanu oczekiwania na rozliczenie.

Rozliczenia

Rozliczenia służą do obsługi faktur i rzeczywistych kwot zakupu.

Przekazanie wniosku do rozliczeń:

1. Otwórz sfinalizowany wniosek.
2. Wybierz przekazanie do rozliczeń.
3. Przejdź do zakładki Rozliczenia.

Dodanie faktury:

1. Otwórz rozliczenie.
2. Dodaj fakturę.
3. Podaj numer faktury.
4. Podaj datę wystawienia.
5. Podaj sprzedawcę i NIP.
6. Podaj kwotę netto i VAT.
7. Wybierz status faktury.
8. Zapisz.

Przypisanie faktury do linii:

1. Otwórz linię rozliczeniową.
2. Wybierz fakturę.
3. Podaj rzeczywistą kwotę brutto.
4. Zapisz.

Zakończenie rozliczenia jest możliwe dopiero wtedy, gdy każda linia rozliczenia ma przypisaną fakturę.

Historia

Po zakończeniu rozliczenia wniosek trafia do historii. Historia służy do podglądu zakończonych spraw. Nie powinna być używana do bieżącego edytowania zakupów.

Najczęstsze sytuacje problemowe

Nie widzę listy zakupów

Możliwe przyczyny:

- lista należy do innego koła,
- lista jest zamknięta,
- lista nie jest powiązana z dostępnym kontekstem zakupowym,
- zalogowany użytkownik nie jest skarbnikiem ani autorem listy.

Nie mogę dodać pozycji do listy

Możliwe przyczyny:

- lista jest zamknięta,
- podano ilość mniejszą lub równą zero,
- przedmiot nie ma poprawnej podkategorii,
- lista należy do nieprawidłowego finansowania.

Nie mogę usunąć pozycji z listy

Możliwe przyczyny:

- lista jest zamknięta,
- próbujesz usunąć pozycję dodaną przez inną osobę,
- nie przekazano identyfikatora zalogowanego użytkownika.

Nie mogę utworzyć wniosku

Możliwe przyczyny:

- dofinansowanie nie należy do wybranej sekcji,
- kwota przekracza dostępne środki dofinansowania,
- kwota przekracza budżet sekcji,
- pozycja planu należy do innego dofinansowania,
- przekroczono plan i nie wpisano uzasadnienia,
- zamknięta lista należy do innego skarbnika.

Nie mogę zakończyć rozliczenia

Możliwe przyczyny:

- rozliczenie nie jest powiązane z wnioskiem,
- co najmniej jedna linia nie ma przypisanej faktury,
- wniosek nie został przekazany do rozliczeń.

Rekomendowany pełny przebieg pracy skarbnika

1. Utwórz albo sprawdź dofinansowanie.
2. Utwórz plan publiczny dla dofinansowania.
3. Dodaj pozycje CPV do planu.
4. Utwórz listę zakupów dla sklepu.
5. Dodaj przedmioty do listy.
6. Zamknij listę.
7. Utwórz wniosek z zamkniętej listy.
8. Przypisz wniosek do dofinansowania i pozycji planu.
9. Sprawdź budżet na pulpicie.
10. Przygotuj finalizację.
11. Zatwierdź finalizację.
12. Przekaż wniosek do rozliczeń.
13. Dodaj faktury.
14. Przypisz faktury do linii rozliczenia.
15. Zakończ rozliczenie.

Uruchomienie i testy

Środowisko uruchomieniowe

Do uruchomienia projektu potrzebne są:

- Docker i Docker Compose,
- Node.js zgodny z wersją obsługiwaną przez Vite,
- npm,
- przeglądarka internetowa.

Backend i baza danych

Uruchomienie backendu i bazy:

```
docker compose up --build
```

Backend działa domyślnie pod adresem:

```
http://localhost:8080
```

Baza PostgreSQL jest wystawiona lokalnie na porcie 5431.

Reset bazy danych

Reset danych wykonuje się przez usunięcie wolumenu Dockera:

```
docker compose down -v  
docker compose up --build
```

Po resecie backend przy starcie tworzy tabele i próbuje załadować dane z pliku:

```
backend/scripts/mockup_data.sql
```

Frontend

Instalacja zależności:

```
cd frontend  
npm install
```

Tryb developerski:

```
npm run dev
```

Build produkcyjny:

```
npm run build
```

Testy backendu

Testy są w katalogu:

```
backend/tests
```

Uruchomienie testów w kontenerze:

```
docker exec -it backend python -m pytest tests/
```

Zakres testów

Obecne testy obejmują między innymi:

- kategorie i podkategorie,
- dodawanie przedmiotów,
- akceptację i odrzucanie przedmiotów,
- izolację danych między kołami,
- budżety i dofinansowania,
- plany publiczne,
- wnioski zakupowe,
- rozliczenia.

Uwagi dotyczące testów

Kody CPV w aktualnym modelu są tekstem. Nowe testy powinny używać wartości typu "31700000" albo "43800000-1", a nie liczb całkowitych.

Generowanie dokumentacji

Dokumentację generuje skrypt:

```
python3 doc_gen.py
```

Wyniki:

```
doc/build/html/index.html  
doc/build/pdf/BudgetFlowFusion_documentation.pdf
```

Skrypt nie wymaga zewnętrznych bibliotek. Generuje statyczny HTML oraz prosty PDF tekstowy.

Struktura testów backendu

```
backend/tests/conftest.py  
    Wspólne fixture: silnik SQLite in-memory, TestClient, mock_db.  
  
backend/tests/test_business_logic.py  
    Kategorie, podkategorie, katalog przedmiotów, akceptacja,  
    odrzucenie i izolacja danych między kołami.  
  
backend/tests/test_public_purchase_plans.py  
    Budżety koła i sekcji, dofinansowania, dashboard, plan publiczny,
```

pozycje CPV, duplikaty CPV i walidacja kwot.

backend/tests/test_purchase_request.py
Pobieranie wniosków, tworzenie wniosku, rezerwacje budżetu, zgodność z planem, przekroczenie planu, alokacje finansowania, pozycje planu i usuwanie wniosku.

backend/tests/test_settlements.py
Lista rozliczeń, filtrowanie po skarbniku, faktury, blokada zakończenia bez faktur i zmiana statusu na settled.

Uruchamianie wybranych testów

Cały pakiet:

```
docker exec -it backend python -m pytest tests/ -v
```

Jeden plik:

```
docker exec -it backend python -m pytest tests/test_purchase_request.py -v
```

Jeden scenariusz:

```
docker exec -it backend python -m pytest  
tests/test_purchase_request.py::test_create_purchase_request -v
```

Szybki tryb po ostatniej awarii:

```
docker exec -it backend python -m pytest tests/ --lf -v
```

Testowa baza danych

Testy nie używają kontenerowej bazy PostgreSQL. confest.py tworzy SQLite in-memory przez:

```
create_engine(  
    "sqlite://",  
    connect_args={"check_same_thread": False},  
    poolclass=StaticPool  
)
```

Następnie `SQLModel.metadata.create_all(engine)` tworzy tabele, a `app.dependency_overrides[get_session]` podmienia sesję backendu na sesję testową.

Konsekwencje:

- testy są szybkie i izolowane,
- nie sprawdzają specyficznych zachowań PostgreSQL,
- migracja `migrate_project_budgets()` nie jest wykonywana w tym samym trybie co w kontenerze produkcyjnym,
- jeśli zmieniasz modele, trzeba sprawdzić zarówno testy SQLite, jak i start aplikacji z PostgreSQL.

Matryca pokrycia reguł biznesowych

Reguła

Testy

Kategorię może utworzyć skarbnik

`test_add_category_and_subcategory`

Katalog przedmiotów jest wspólny i widoczny po dodaniu

`test_normal_student_adds_item_is_visible_in_catalog`

`test_treasurer_adds_item_is_approved`

Skarbnik nie widzi pending itemów obcego koła

`test_multitenancy_isolation_for_treasurer`

Budżet koła jest sumą budżetów sekcji/dofinansowań

`test_get_association_budgets_with_fundings`

`test_dashboard_budget_is_sum_of_section_fundings`

Dofinansowanie jest przypisane do jednej sekcji

`test_funding_is_assigned_to_one_section_budget`

Plan publiczny jest jeden na dofinansowanie

`test_public_purchase_plan_list_update_keeps_single_plan_per_funding`

CPV w planie nie może się dublować

`test_create_public_purchase_plan_rejects_duplicate_cpv`

Kwota pozycji planu musi być dodatnia

`test_create_public_purchase_plan_rejects_non_positive_cost`

Wniosek rezerwuje środki budżetu i dofinansowania

`test_create_purchase_request`

Przekroczenie planu wymaga uzasadnienia

`test_create_purchase_request_rejects_plan_overrun_without_justification`

Przekroczenie planu z uzasadnieniem przechodzi jako requires_approval

`test_create_purchase_request_allows_plan_overrun_with_justification`

Wniosek zapisuje wiele alokacji finansowania

`test_create_purchase_request_stores_funding_allocations`

Wniosek zapisuje pozycje planu powiązane z koszykiem

`test_create_purchase_request_stores_plan_positions_when_shop_list_is_provided`

Rozliczenia wymagają faktur na liniach

`test_complete_settlement_rejects_line_without_invoice`

Zakończenie rozliczenia ustawia status settled

`test_update_settlement_status_marks_request_as_settled`

Checklist przed oddaniem zmian

1. Uruchomić testy backendu.
2. Uruchomić `npm run build` we frontendzie.
3. Odpalić `docker compose up --build` i sprawdzić start backendu.
4. Sprawdzić logowanie zwykłego użytkownika i skarbnika.

5. Sprawdzić scenariusz: dofinansowanie -> plan publiczny -> koszyk -> wniosek -> finalizacja -> rozliczenie -> faktura.
6. Wygenerować dokumentację przez python3 doc_gen.py.

Utrzymanie i słownik pojęć

Zasady utrzymania

1. Zmiany w modelach SQLAlchemy powinny być odzwierciedlone w funkcji migracyjnej w `backend/src/__init__.py`.
2. Zmiany w endpointach powinny być odzwierciedlone w dokumentacji API.
3. Zmiany w procesie użytkownika powinny aktualizować dokumentację użytkową.
4. Po zmianach finansowych należy sprawdzić dashboard budżetowy, listę dofinansowań, tworzenie wniosku i rozliczenia.
5. Przy zmianach w planach publicznych należy sprawdzić wykorzystanie pozycji planu oraz finalizację wniosku.

Miejsca wysokiego ryzyka

Budżety Kwoty są liczone z kilku źródeł: dofinansowań, wydanych pieniędzy, alokacji wniosków i finalizacji. Należy unikać podwójnego liczenia.

Plany publiczne Pozycja planu może być wykorzystana przez wiele wniosków. Usunięcie albo zmiana pozycji wpływa na raportowanie i zgodność z planem.

Finalizacja Snapshot powinien chronić historię dokumentu. Nie należy usuwać snapshotów bez świadomej migracji.

Rozliczenia Faktury i linie rozliczeń są powiązane z wnioskiem. Zmiana statusów powinna zachować spójność historii.

Role Frontend ukrywa część funkcji, ale backend także powinien pilnować dostępu tam, gdzie operacja wpływa na finanse.

Słownik pojęć

Koło naukowe Organizacja użytkowników systemu. W modelu reprezentowana przez `Association`.

Członek koła Zwykły użytkownik, który może dodawać przedmioty i pracować na dostępnych koszykach.

Skarbnik Użytkownik z powiązaniem `ProjectFinanceManager`. Ma dostęp do finansów, planów, wniosków i rozliczeń.

Sekcja / projekt Część koła realizująca określony projekt. W modelu jest to `Project` oraz jego `ProjectBudget`.

Budżet koła Zbiorczy budżet na poziomie `AssociationBudget`.

Budżet sekcji Budżet projektu, reprezentowany przez `ProjectBudget`.

Dofinansowanie Konkretnie źródło pieniędzy, reprezentowane przez `Funding`.

Zadanie budżetowe Część dofinansowania przypisana do konkretnego celu, reprezentowana przez `FundingTask`.

Plan zamówień publicznych Roczny plan dla jednego dofinansowania, reprezentowany przez `PublicPurchasePlanList`.

Pozycja planu publicznego Wiersz planu określający CPV i kwotę, reprezentowany przez `PublicPurchasePlan`.

CPV Kod klasyfikacji zamówień publicznych. W systemie używany do grupowania planów i

zakupów.

Koszyk sklepowy Lista zakupów dla jednego sklepu, reprezentowana przez ShopPurchaseList.

Wniosek o zamówienie Formalny dokument zakupowy, reprezentowany przez PurchaseRequest.

Alokacja finansowania Przypisanie części kwoty wniosku do dofinansowania, reprezentowane przez PurchaseRequestFundingAllocation.

Alokacja planu Przypisanie części kwoty wniosku do pozycji planu publicznego, reprezentowane przez PurchaseRequestPlanPosition.

Finalizacja Etap zamknięcia danych zakupowych wniosku i przygotowania dokumentu do księgowości.

Snapshot finalizacji Historyczny zapis danych planu i finansowania, reprezentowany przez PurchaseRequestFinalizationSnapshot.

Rozliczenie Etap obsługi faktur i rzeczywistych kwot, reprezentowany przez Settlement.

Faktura Dokument księgowy, reprezentowany przez Invoice.

Linia rozliczeniowa Pozycja w rozliczeniu wniosku, reprezentowana przez PurchaseRequestSettlementLine.

Rekomendowane dalsze usprawnienia

- Dodać pełniejsze testy procesu finalizacji i rozliczeń.
- Ujednolicić typ CPV we wszystkich testach i formularzach jako tekst.
- Rozważyć wydzielenie warstwy serwisów z endpointów.
- Dodać jawne uprawnienia backendowe dla operacji skarbnika.
- Dodać eksport dokumentów wniosku do oficjalnego szablonu.

Checklist techniczny przy zmianie modelu danych

1. Zmienić klasę SQLAlchemyModel w backend/src/relations.py.
2. Jeżeli aplikacja ma działać na istniejącej bazie, dopisać migrację w migrate_project_budgets().
3. Sprawdzić, czy nowe pole wymaga aktualizacji modeli Pydantic w backend/src/routes.
4. Sprawdzić, czy frontend wysyła albo czyta nowe pole.
5. Zaktualizować testy fixture, szczególnie conftest.py i helpery w plikach testowych.
6. Uruchomić testy backendu i start kontenerów.
7. Zaktualizować dokumentację modelu danych i API.

Checklist techniczny przy zmianie budżetów

Zmiana finansowa powinna zostać sprawdzona w tych miejscach:

```
backend/src/routes/public_purchase_plans_routes.py
    _project_budget_amounts
    _association_budget_amounts
    _budget_out
    get_dashboard_budget_summary

backend/src/routes/fundings_routes.py
    _funding_out
```

```
backend/src/routes/purchase_request_routes.py
    _budget_info_for_request
    create_purchase_request
    update_purchase_request
    finalize_purchase_request
```

Minimalny scenariusz ręczny:

1. Utworzyć dofinansowanie w sekcji.
2. Sprawdzić, czy budżet sekcji zwiększa się o kwotę dofinansowania.
3. Sprawdzić, czy budżet koła jest sumą sekcji.
4. Utworzyć wniosek z tego dofinansowania.
5. Sprawdzić, czy `purchase_requests_total_allocated` rośnie.
6. Sprawdzić, czy `available_after_purchase_requests` maleje na poziomie dofinansowania, sekcji i koła.

Checklist techniczny przy zmianie planów publicznych

Miejsca implementacji:

```
PublicPurchasePlanList
PublicPurchasePlan
PurchaseRequestPlanPosition
GroupedShopsListByCpvCategoryAndFunding
ProductCategory.public_purchase_plan_id
```

Reguły do zachowania:

- jeden `PublicPurchasePlanList` na `Funding`,
- unikalny `cpv_code` w ramach jednego planu,
- dodatni `cost`,
- `remaining_amount = cost - used_amount`,
- brak możliwości usunięcia pozycji użytej we wniosku,
- brak możliwości usunięcia pozycji z podpętymi koszykami.

Checklist techniczny przy zmianie wniosków

Miejsca implementacji:

```
PurchaseRequest
PurchaseRequestFundingAllocation
PurchaseRequestPlanPosition
PurchaseRequestFinalizationSnapshot
PurchaseRequestSettlementLine
```

Reguły do zachowania:

- `funding_id` musi należeć do `project_budget_id`,
- pozycja planu musi należeć do jednego z finansowań użytych we wniosku,
- przekroczenie planu wymaga uzasadnienia,
- brak pozycji planu oznacza draft,
- finalizacja wymaga koszyków i pozycji CPV,

- finalizacja blokuje dalsze dodawanie przez `can_add = False`,
- finalizacja zapisuje snapshot,
- przekazanie do rozliczeń ustawia `finalization_status = settlement`.

Checklist techniczny przy zmianie list zakupów

Reguły do zachowania:

- lista otwarta ma `settlement_id = NULL`,
- lista zamknięta ma `settlement_id` i nie pozwala zmieniać pozycji,
- `ShopPurchaseListItem.amount` jest sumą wkładów,
- `ShopPurchaseListItemContribution` decyduje o tym, ile może usunąć zwykły student,
- skarbnik może zamknąć tylko listę utworzoną przez siebie,
- duplikat sklepu w tej samej grupie `gslbccf_id` jest blokowany.

Ryzyka techniczne

Brak pełnej autoryzacji serwerowej Frontend zapisuje użytkownika w `localStorage`. Backend wykonuje wybrane walidacje, ale nie ma centralnego middleware autoryzacji.

Logika biznesowa w endpointach Duża część reguł jest w funkcjach tras. Przy rozbudowie projektu warto wydzielić warstwę serwisów i testować ją niezależnie od HTTP.

Migracje runtime `migrate_project_budgets()` wykonuje ręczne `ALTER TABLE` przy starcie aplikacji. To wygodne w projekcie akademickim, ale w systemie produkcyjnym lepsze byłyby wersjonowane migracje.

SQLite w testach i PostgreSQL w runtime Testy używają SQLite in-memory, a aplikacja działa na PostgreSQL. Trzeba uważać na różnice w enumach, typach dat, ograniczeniach FK i składni SQL.

Kwoty netto/brutto Plan publiczny działa na kwotach netto, koszyki i rozliczenia na kwotach brutto. Finalizacja przelicza wartości przez 1.23. Zmiana VAT albo walut powinna objąć cały przepływ.

CPV jako tekst CPV ma format tekstowy i może zawierać myślnik. Nie należy używać `int` w nowych polach, testach ani payloadach.

Sugestia przyszłej refaktoryzacji

Najbardziej opłacalne wydzielenie warstwy serwisowej:

```
services/budget_service.py
    liczenie budżetu sekcji, koła i dofinansowań

services/public_plan_service.py
    użycie planu, remaining_amount, walidacja CPV

services/purchase_request_service.py
    tworzenie wniosku, alokacje, status zgodności

services/finalization_service.py
    snapshot, netto/brutto, główny CPV

services/settlement_service.py
    linie rozliczeń, faktury, zamknięcie sprawy
```

Po takim wydzieleniu endpointy powinny głównie mapować HTTP na wywołania serwisów, a większość reguł można byłoby testować bez TestClient.